TP N°2.2:

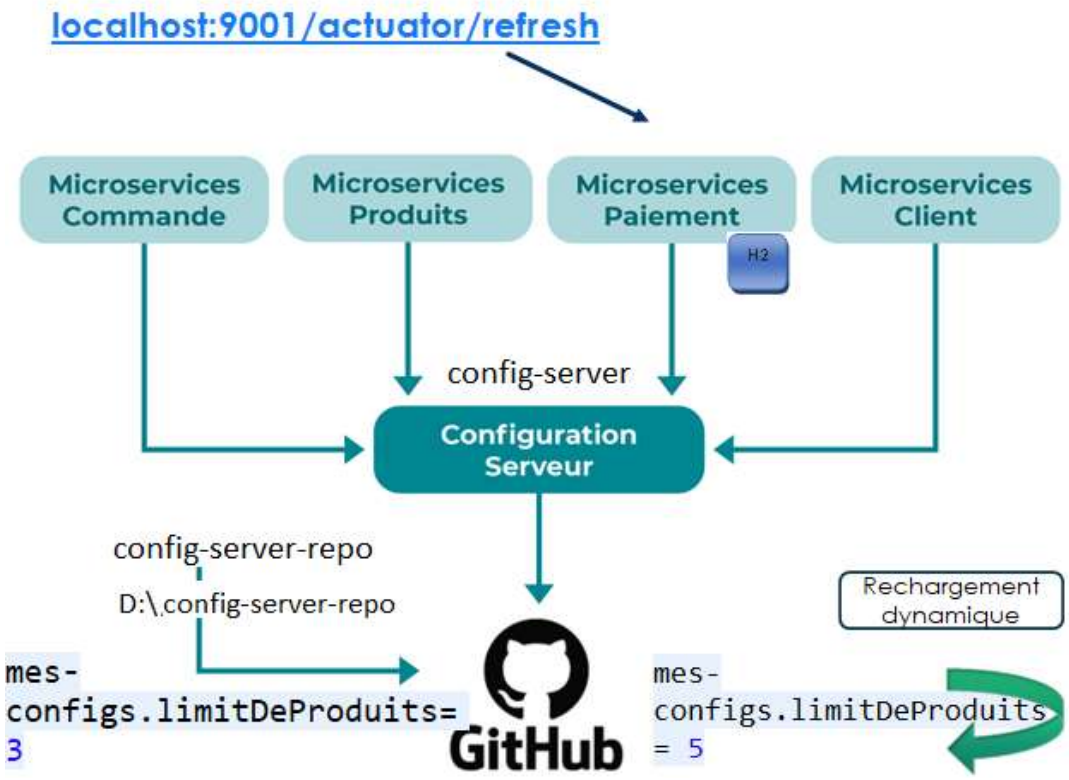
Monitoring des microservices : Spring Cloud Actuator

1. Prérequis
   - TP3.1 Spring Cloud Config : ce n'est pas nécessaire d'avoir le Spring Cloud Config Server
   - POSTMAN ou un autre outil pour tester les méthodes POST, PUT et DELETE.

2. Objectifs

   1. Développer un microservice « Monitorable » grâce à Spring Actuator
   2. Afficher les services offerts par Spring Actuator
   3. Mise en œuvre du service Spring Actuator
   4. Rechargement dynamique de la configuration d'un microservice : @RefreshScope
   5. Etat de santé du microservice avec le service «health»
   6. Personnalisation du service health avec l'interface « HealthIndicator »

3. Architecture de mise en œuvre



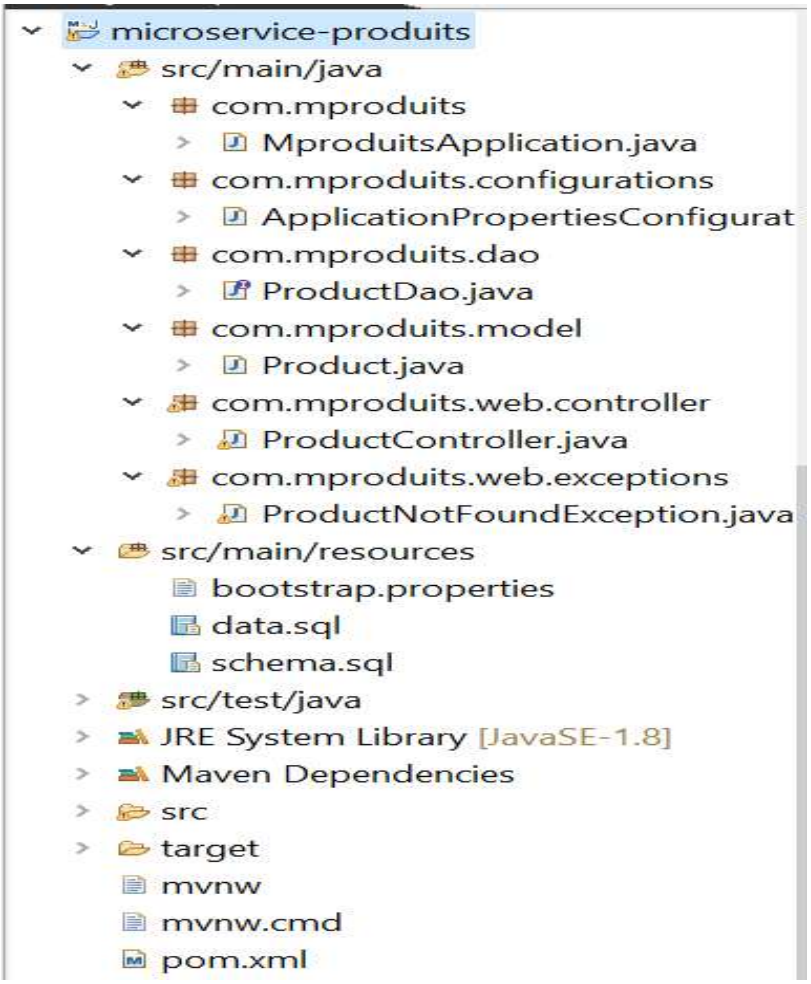
4. Démarche de développement de l'application :

1. Use case : Le microservice « Produits » permet de réaliser les opération CRUD et permet en outre d'afficher la liste des produits avec une taille limite configurée au niveau du fichier de configuration : « `mes-configs.limitDeProduits= 4` »
2. On va modifier cette propriété à « 5 » puis appliquer le service Actuator « refresh » du microservice-produit.
3. Vérifier que la configuration a été prise en charge de manière dynamique (à chaud).

5. Développement du microservice « Produit »

a. Se baser sur le projet microservice-produit dans sa version de configuration locale ou bien avec Spring cloud Config Server.

b. Dans ce TP on va se baser sur la version avec Spring Cloud Config Server



c. « bootstrap.properties » du projet du microservice-produits:

```
spring.application.name=microservice-produits
#URL de Spring Cloud Config Server
spring.cloud.config.uri=http://localhost:9101
```

d. « microservice-produits.properties»

```
#Configurations H2
spring.jpa.show-sql=true
spring.h2.console.enabled=true
spring.datasource.sql-script-encoding=UTF-8
#Les configurations exetrenalisés
mes-configs.limitDeProduits= 4
#Configuration Actuator
management.endpoints.web.exposure.include=*
#management.endpoints.web.exposure.include=refresh
```

- La configuration d'Actuator se fait via la ligne :

  `management.endpoints.web.exposure.include=*` et permet d'expose l'ensemble des services Actuator disponibles (~24).

- «`=refresh`» Indique aux beans de se rafraichir à chaque fois qu'un événement Refresh est lancé

- Actuator expose des endpoints/API qui fournissent des données sur des aspects du microservice

- Si on utiliser Spring Cloud Config : Ne pas oublier de synchroniser le repository local avec Github.

e. Vérifier que le starter « `spring-boot-starter-actuator` » existe au niveau du fichier pom.xml :

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
	xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
	<modelVersion>4.0.0</modelVersion>
	<groupId>com.mproduits</groupId>
	<artifactId>mproduits</artifactId>
	<version>0.0.1-SNAPSHOT</version>
	<packaging>jar</packaging>
	<name>mproduits</name>
	<description>Microservice de gestion des produits</description>
	<parent>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-parent</artifactId>
		<version>2.7.16</version>
		<relativePath/>
	</parent>
	<properties>
		<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
		<project.reporting.outputEncoding>UTF-
8</project.reporting.outputEncoding>
		<java.version>1.8</java.version>
```

```
		<spring-cloud.version>2021.0.8</spring-cloud.version>
	</properties>
	<dependencies>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-web</artifactId>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-data-jpa</artifactId>
		</dependency>
		<dependency>
			<groupId>com.h2database</groupId>
			<artifactId>h2</artifactId>
			<scope>runtime</scope>
		</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-configuration-
processor</artifactId>
			<optional>true</optional>
		</dependency>

		<dependency>
			<groupId>org.springframework.cloud</groupId>
			<artifactId>spring-cloud-starter-config</artifactId>
		</dependency>

<!-- Add the following dependency to avoid : No spring.config.import property
has been defined -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-bootstrap</artifactId>
</dependency>

<dependency>
	<groupId>org.springframework.boot</groupId>
	 <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
	<groupId>org.springframework.cloud</groupId>
	<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
		<dependency>
			<groupId>org.springframework.boot</groupId>
			<artifactId>spring-boot-starter-test</artifactId>
			<scope>test</scope>
		</dependency>
	</dependencies>

	<dependencyManagement>
		<dependencies>
			<dependency>
				<groupId>org.springframework.cloud</groupId>
				<artifactId>spring-cloud-dependencies</artifactId>
				<version>${spring-cloud.version}</version>
				<type>pom</type>
```

```
                        <scope>import</scope>
                    </dependency>
                </dependencies>
        </dependencyManagement>

        <build> <plugins>  <plugin>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin></plugins></build>
        <repositories>
                <repository>
                        <id>spring-milestones</id>
                        <name>Spring Milestones</name>
                        <url>https://repo.spring.io/milestone</url>
                        <snapshots>
                                <enabled>false</enabled>
                        </snapshots>
                </repository>
        </repositories>
</project>
```

```
package com.mproduits.web.controller;

import com.mproduits.configurations.ApplicationPropertiesConfiguration;
import com.mproduits.dao.ProductDao;
import com.mproduits.model.Product;
import com.mproduits.web.exceptions.ProductNotFoundException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.cloud.context.config.annotation.RefreshScope;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
import java.util.Optional;

@RestController
public class ProductController implements HealthIndicator {

        @Autowired
        ProductDao productDao;

        @Autowired
        ApplicationPropertiesConfiguration appProperties;

        // Affiche la liste de tous les produits disponibles
        @GetMapping(value = "/Produits")
        public List<Product> listeDesProduits() {

                System.out.println(" ********* ProductController listeDesProduits() ");
                List<Product> products = productDao.findAll();

                if (products.isEmpty())
                        throw new ProductNotFoundException("Aucun produit n'est disponible
à la vente");

                List<Product> listeLimitee = products.subList(0,
appProperties.getLimitDeProduits());

                return listeLimitee;
```

```
	}

	// Récuperer un produit par son id
	@GetMapping(value = "/Produits/{id}")
	public Optional<Product> recupererUnProduit(@PathVariable int id) {
		System.out.println(" ********* ProductController
recupererUnProduit(@PathVariable int id) ");

		Optional<Product> product = productDao.findById(id);

		if (!product.isPresent())
			throw new ProductNotFoundException("Le produit correspondant à
l'id " + id + " n'existe pas");

		return product;
	}

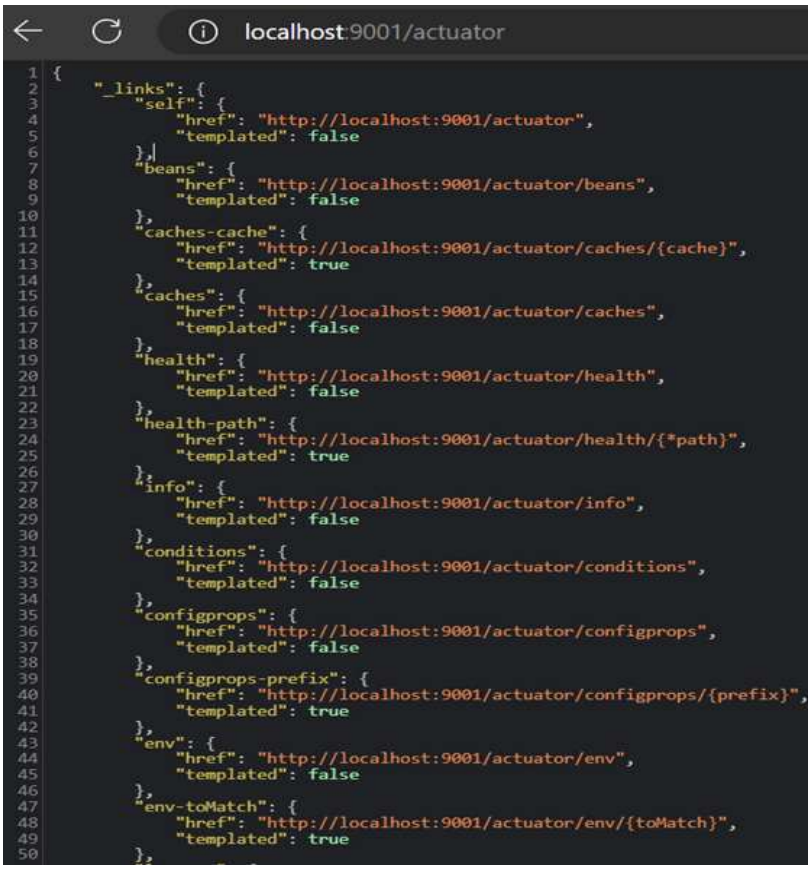
	@Override
	public Health health() {

		System.out.println("****** Actuator : ProductController health() ");

		List<Product> products = productDao.findAll();
		if (products.isEmpty()) {
			return Health.down().build();
		}
		return Health.up().build();
	}
}
```

f. http://localhost:9001/actuator

Dans le cas où : `management.endpoints.web.exposure.include=*`

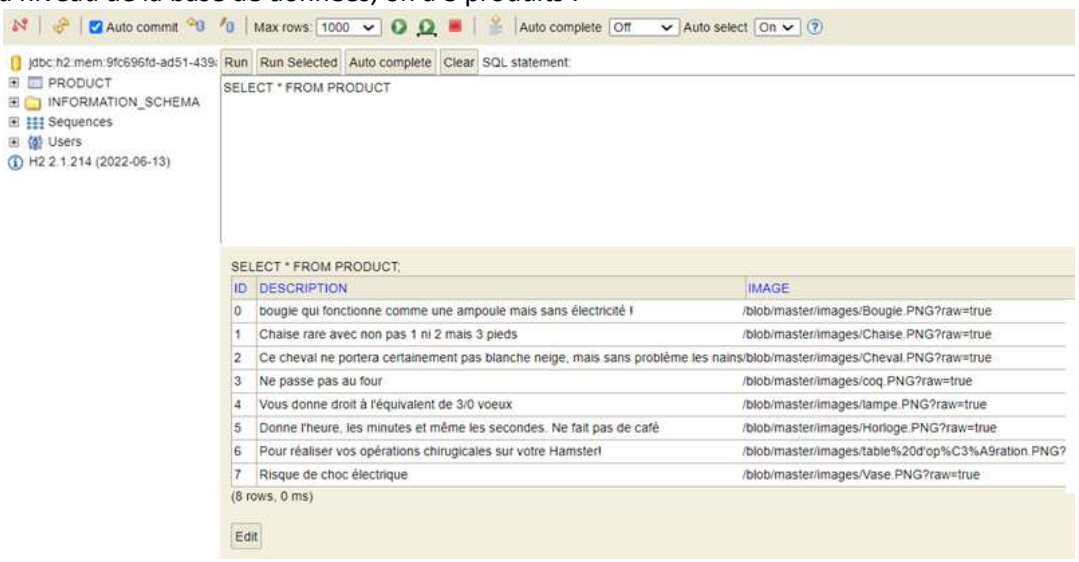



```
        "loggers": {
            "href": "http://localhost:9001/actuator/loggers",
            "templated": false
        },
        "loggers-name": {
            "href": "http://localhost:9001/actuator/loggers/{name}",
            "templated": true
        },
        "heapdump": {
            "href": "http://localhost:9001/actuator/heapdump",
            "templated": false
        },
        "threaddump": {
            "href": "http://localhost:9001/actuator/threaddump",
            "templated": false
        },
        "metrics": {
            "href": "http://localhost:9001/actuator/metrics",
            "templated": false
        },
        "metrics-requiredMetricName": {
            "href": "http://localhost:9001/actuator/metrics/{requiredMetricName}",
            "templated": true
        },
        "scheduledtasks": {
            "href": "http://localhost:9001/actuator/scheduledtasks",
            "templated": false
        },
        "mappings": {
            "href": "http://localhost:9001/actuator/mappings",
            "templated": false
        },
        "refresh": {
            "href": "http://localhost:9001/actuator/refresh",
            "templated": false
        },
        "features": {
            "href": "http://localhost:9001/actuator/features",
            "templated": false
        },
        "serviceregistry": {
            "href": "http://localhost:9001/actuator/serviceregistry",
            "templated": false
        }
    }
}
```

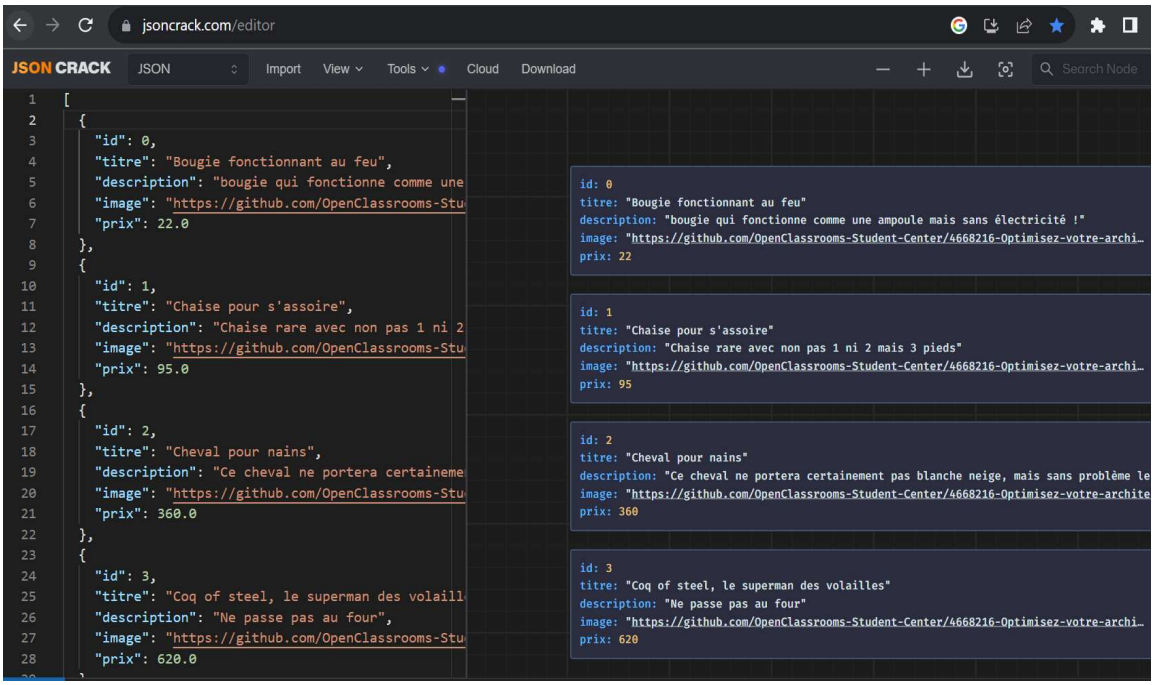
➔ Dans le cas où : `management.endpoints.web.exposure.include=refresh`



g. Vérifier le bon fonctionnement du MS-Produits :
Au niveau de la base de données, on a 8 produits :



➔ http://localhost:9001/Produits ➔ Affichage de 4 produits

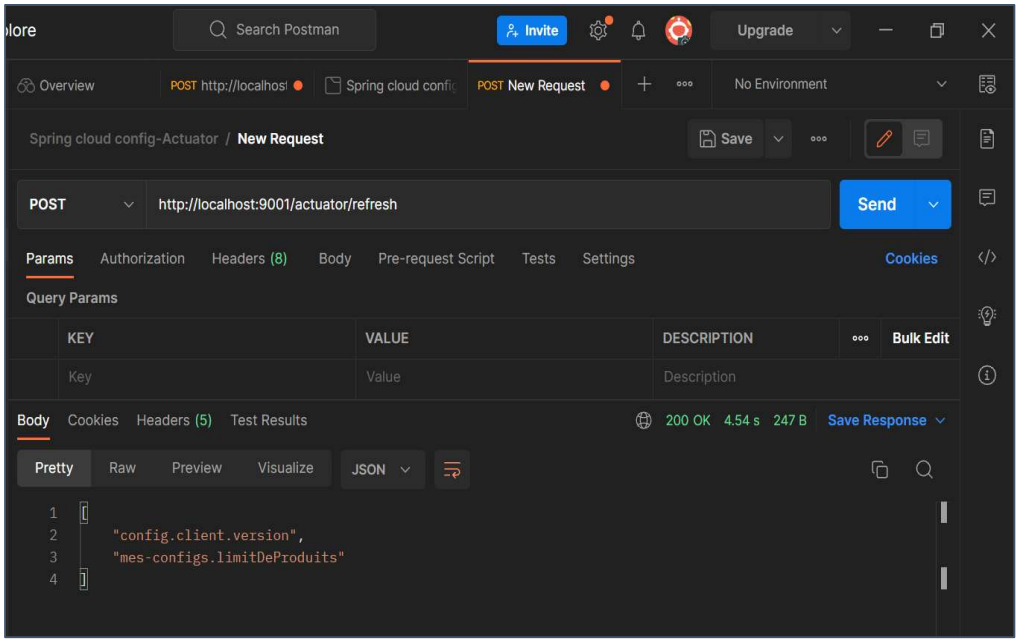


6. Rechargement de la configuration à chaud

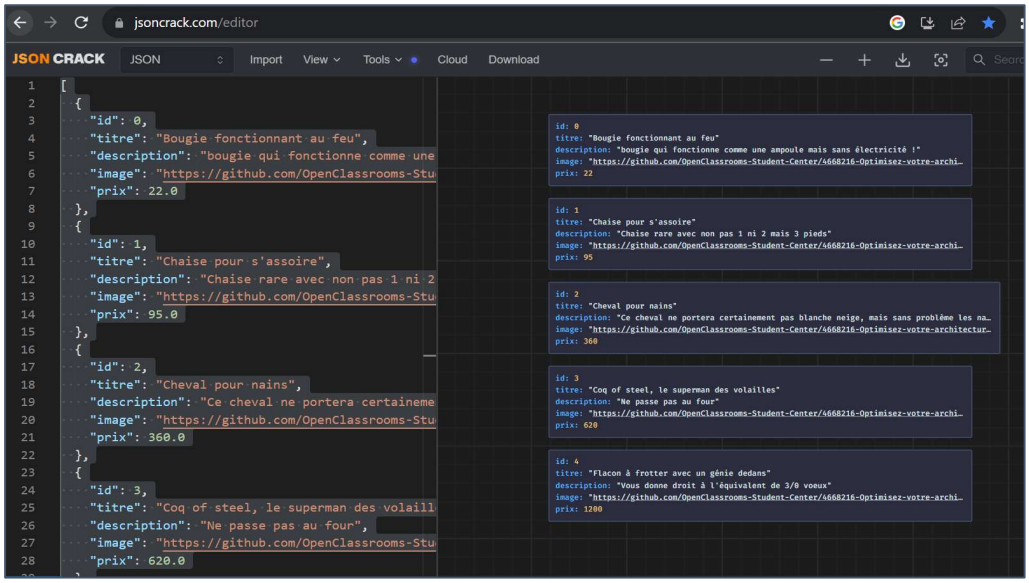
a. Modifier la propriété `mes-configs.limitDeProduits= 5`: augmenter le nombre à 5 (Commit puis push) → rien ne se passe

Déclenchez un événement Refresh en envoyant une requête POST http://localhost:9001/actuator/refresh via Postman qui indique exactement les propriétés qui ont été modifiées depuis le dernier commit

b. Remarque Request method 'GET' not supported] : il faut utiliser la méthode POST au lieu de GET



c. http://localhost:9001/Produits → Affichage de 5 produits

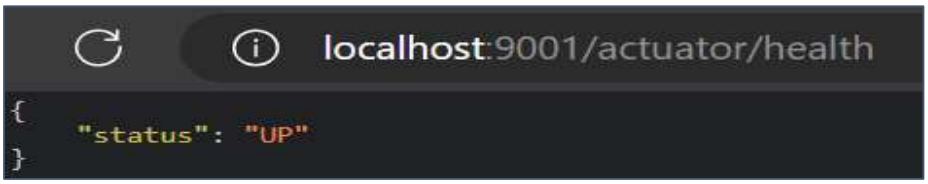


7. Etat de santé du microservice : service « health »

a. Grâce à l'interface « HealthIndicator » :

```
@RestController
public class ProductController implements HealthIndicator
```

b. On peut tester l'état de santé du MS : localhost:9001/actuator/health



c. Personnalisation du comportement de « HealthIndicator » :

```
@Override
public Health health()
{
List<Product> products = productDao.findAll();
if(products.isEmpty())
{
   return Health.down().build();
}
return Health.up().build();
```

